Contents

1.Introduction: 2
2. Project Overview: 2
3. Project Objectives: 2
4. Development Approach: 3
5. Ten-Week Development Plan: 3
6. System Requirements: 1. Hardware Requirements 4
7. Main Features: 4
8. Project Structure: 5
9. Object-Oriented Programming Concepts: 1. Class and Object 5
10. Game Rules and Logic: 6
11.Important Code Logic: 6
12. Graphical User Interface: 7
13.Gameplay Procedure: 7
14. How to Download the project from Github: 8
15. Testing 9
16. Challenges Faced: 9
17. Limitations: 9
18. Future Improvements 10
19. Result and Discussion: 10
20. Conclusion: 11

1.Introduction:

The Snake and Ladder Game is a Java-based graphical desktop application developed as a Software Development project. The main purpose of the project is to create an interactive board game in which two players take turns rolling a dice and moving their tokens from the starting position toward position 100.

The application uses Java Swing to build the graphical user interface. Object-Oriented Programming concepts are applied by separating player information, dice generation, board information and game control into different classes. The project is developed incrementally so that the graphical board, player movement, Snake and Ladder rules, and winning condition can be implemented and tested step by step. The game is designed as a simple desktop application without a database. Its focus is on GUI development, event handling, class-based design, random number generation and game logic.

2. Project Overview:

The game contains a 10×10 board with positions from 1 to 100. Two players participate in the game. Each player starts from position 0 and moves according to the number obtained from the dice. Several cells contain snakes and ladders. When a player lands on a ladder cell, the player moves upward to the corresponding destination. When a player lands on a snake cell, the player moves downward to the corresponding destination. The first player to reach position 100 wins the game.

Item	Description
Application Type	Java Swing desktop game
Players	Two players
Board	10×10, positions 1–100
Input	ROLL DICE and RESET GAME buttons
Movement	Dice-based player movement
Special Cells	Snakes and ladders
Winning Point	Position 100
Database	Not used

3. Project Objectives:

- To develop an interactive Snake and Ladder game using Java.
- To create a simple graphical board using Java Swing.
- To display positions from 1 to 100 in a 10×10 layout.
- To implement two-player turn-based gameplay.

- To generate dice values randomly from 1 to 6.
- To display player positions and graphical player tokens.
- To implement Snake and Ladder movement rules.
- To prevent a player from moving beyond position 100.
- To detect the winning condition when a player reaches position 100.
- To provide a reset option for starting a new game.
- To apply Object-Oriented Programming concepts through separate classes.
- To practice GUI event handling and game-state management.

4. Development Approach:

The project follows an incremental development approach. Instead of implementing the complete game at once, the system was divided into smaller tasks. Each week focused on a particular part of the project, followed by testing before moving to the next stage.

This approach matches the project's development plan because the board and interface were created first, followed by dice and turn logic, graphical player movement, Snake and Ladder behavior, and finally winning, testing and documentation.

5. Ten-Week Development Plan:

10 weeks project plan for Snake and Ladder Game

Week	Activities
Week 1	Project requirements and objectives will be identified. The game rules, features, and scope will be analyzed and documented.
Week 2	The overall system design and game flow will be prepared. GUI layout and class structure will be planned.
Week 3	The development environment will be set up and the basic project structure will be created using Java and Swing.
Week 4	The graphical user interface, including the game board, buttons, labels, and player tokens, will be designed and implemented.
Week 5	The virtual dice rolling functionality will be developed using random number generation techniques.
Week 6	Player movement logic will be implemented, allowing players to move according to the dice value and switch turns automatically.
Week 7	Snake and ladder positions will be integrated, and automatic position updates will be implemented.
Week 8	Winning conditions and game completion logic will be added along with user notifications.
Week 9	The application will be tested thoroughly to identify and fix bugs and improve performance.
Week 10	Final documentation, project report, screenshots, and presentation slides will be prepared for submission.

6. System Requirements:

1. Hardware Requirements

- A standard laptop or desktop computer
- At least 4 GB RAM recommended
- Keyboard and mouse
- Monitor/display

2. Software Requirements

- Windows operating system
- Java Development Kit (JDK)
- NetBeans IDE
- Java Swing libraries included with Java

7. Main Features:

Feature	Implementation
Graphical Game Board	A 10×10 board displays positions 1 to 100.
Two Players	Player 1 and Player 2 take turns during the game.
Dice Rolling	The dice generates a random value between 1 and 6.
Position Display	The current position of each player is shown in the game information area.
Graphical Tokens	Player 1 and Player 2 are represented on their current cells.
Snake Movement	Landing on a snake moves the player to the snake's lower position.
Ladder Movement	Landing on a ladder moves the player to the ladder's upper position.
Turn Management	The current player's turn is displayed and changed after each move.
Boundary Check	A move that would go beyond 100 is not performed.
Winning Detection	Reaching position 100 produces a winning message and ends rolling.
Reset	The game can be restarted from the initial state.

8. Project Structure:

The final implementation is organized into separate Java classes. The current codebase uses four main classes. The graphical board panel and game controls are managed inside MainGame rather than a separate GamePanel class.

Class	Responsibility
MainGame.java	Creates the GUI, handles buttons, turns, player movement, board updates, Snake/Ladder effects and winning.
Player.java	Stores player name and current position and provides position/reset methods.
Dice.java	Generates a random dice value from 1 to 6.
Board.java	Stores Snake/Ladder mappings and checks the destination of a special cell.

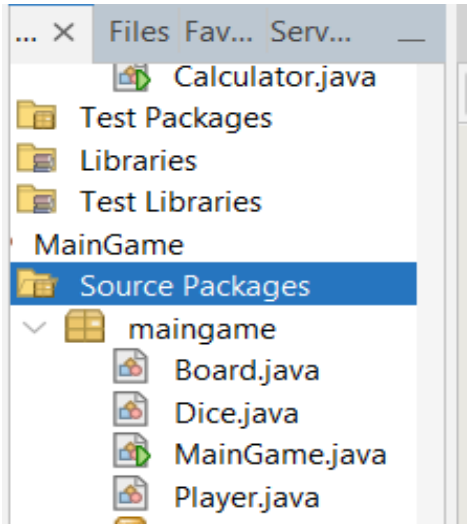


Fig: project structure

9. Object-Oriented Programming Concepts:

1. Class and Object

The project uses separate classes such as Player, Dice, Board and MainGame. Objects are created from these classes to represent the game components.

2. Encapsulation

Player data such as name and position are kept private. Access is provided through methods such as getPosition(), setPosition(), getName() and reset(). This keeps the internal data controlled by the class.

3. Modularity

Different responsibilities are divided among classes. For example, Dice handles dice generation while Board manages Snake and Ladder mappings. This makes the code easier to understand and maintain.

4. Event Handling

Swing action listeners connect GUI buttons with game methods. For example, pressing the ROLL DICE button calls rollDice(), while pressing RESET GAME calls resetGame().

10. Game Rules and Logic:

1. Both players start from position 0.
2. Players take turns rolling the dice.
3. The dice produces a value from 1 to 6.
4. The dice value is added to the current position.
5. If the resulting position is greater than 100, the player does not move.
6. If the player lands on a ladder, the position changes to the ladder's destination.
7. If the player lands on a snake, the position changes to the snake's destination.
8. After a normal completed turn, control passes to the other player.
9. The player who reaches position 100 wins.
10. After a win, the Roll Dice button is disabled until the game is reset.

11.Important Code Logic:

1.Dice Generation

The Dice class contains the rollDice() method. It generates a random integer between 1 and 6 and returns the value to MainGame.

2. Player Position Update

After a dice roll, the current player's old position is added to the dice value. The resulting position is checked before it is stored in the Player object.

3. Snake and Ladder Check

The Board class provides isSnake(), isLadder() and checkPosition() methods. MainGame uses these methods after a player lands on a cell. A ladder changes the player to its upper destination, while a snake changes the player to its lower destination.

4. Board Refresh

After a player's position changes, updateBoard() refreshes all 100 cells. updateCell() checks whether the current cell contains a Snake, Ladder or player token and updates the displayed content.

5. Winner Detection

After Snake/Ladder processing, the program checks whether the current player's final position is 100. If it is, a Game Over dialog is displayed, a winning status is shown and the Roll Dice button is disabled.

6. Reset

The resetGame() method returns both players to position 0, sets the turn back to Player 1, clears the dice display, refreshes the board and enables the Roll Dice button.

12. Graphical User Interface:

The GUI is developed with Java Swing. The main window contains a title, the 10×10 game board and a game information panel. The information panel displays player positions, dice result, current turn and game status.

The board cells are implemented using JLabel components. Normal cells use a light background, Snake cells use a different background, and Ladder cells use another background. Player indicators are displayed on the corresponding cell.

GUI Component	Purpose
JFrame	Main application window.
JPanel	Board and control panel containers.
JLabel	Board cells, positions, status and player indicators.
JButton	ROLL DICE and RESET GAME controls.
GridLayout	10×10 board arrangement.
BorderLayout	Main window arrangement.

13.Gameplay Procedure:

1. Run MainGame.java from NetBeans.
2. The Snake and Ladder board appears with the game information panel.
3. Player 1 starts the game.
4. Click ROLL DICE to generate a value from 1 to 6.
5. The player's position and token are updated.
6. If the player lands on a ladder, the player moves upward.
7. If the player lands on a snake, the player moves downward.
8. The turn changes to the other player.
9. The same process continues until a player reaches position 100.
10. The winning message is displayed and further dice rolling is disabled.
11. Click RESET GAME to start again.

14. How to Download the project from Github:

Step 1: Open the GitHub Repository The user first needs to open the project's GitHub repository.

Step 2: Click the Code Button After opening the repository, click the Code button.

Step 3: Select Download ZIP From the Code menu, select Download ZIP. GitHub will download the project as a ZIP file.

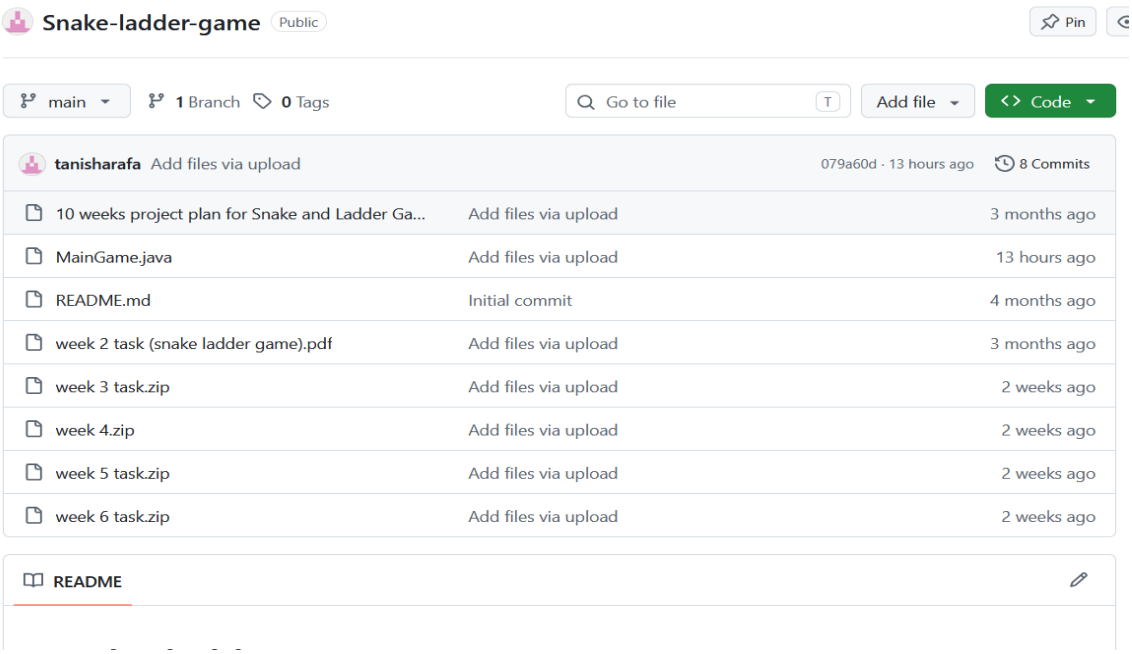
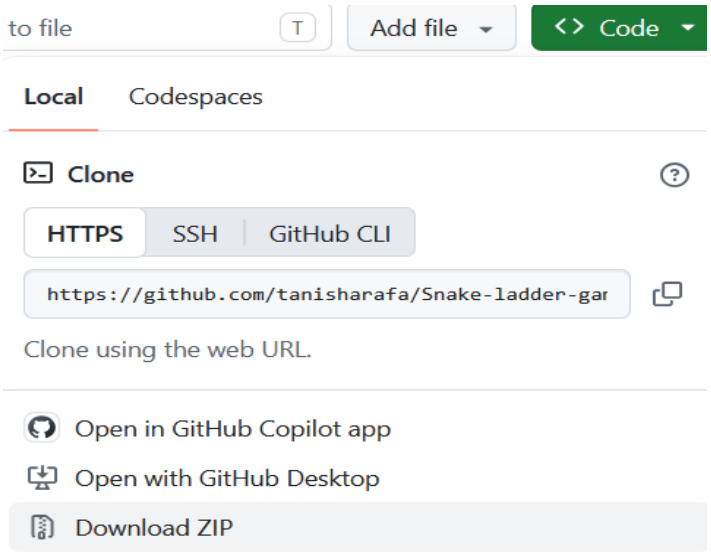


Fig: Github Respiratory

Step 4: Extract the ZIP File After downloading, extract the ZIP file. The extracted folder will contain the project files.



Downloading project from Github

15. Testing

Test Case	Expected Result	Status
Application start	Game window and board appear.	Passed
Dice roll	A value from 1 to 6 is displayed.	Passed
Player movement	Current player position changes according to dice.	Passed
Turn change	Control switches between Player 1 and Player 2.	Passed
Move beyond 100	Player remains in the previous position.	Passed
Ladder landing	Player moves to ladder destination.	Passed
Snake landing	Player moves to snake destination.	Passed
Player token	Token indicator appears on the current cell.	Passed
Winning	Position 100 displays a winning message.	Passed
Game over	Roll Dice is disabled after winning.	Passed
Reset	Both players return to position 0 and Player 1 starts.	Passed

16. Challenges Faced:

- Creating the 10×10 board with the required 1–100 zig-zag numbering.
- Connecting Swing buttons with game methods.
- Keeping player positions synchronized with the visual board.
- Refreshing the board after every dice roll.
- Implementing Snake and Ladder destination checking.
- Managing two-player turn switching.
- Preventing movement beyond position 100.
- Handling the winning condition and disabling further rolls.
- Dealing with Unicode/emoji display differences in Swing fonts.
- Testing different dice results and special-cell cases.

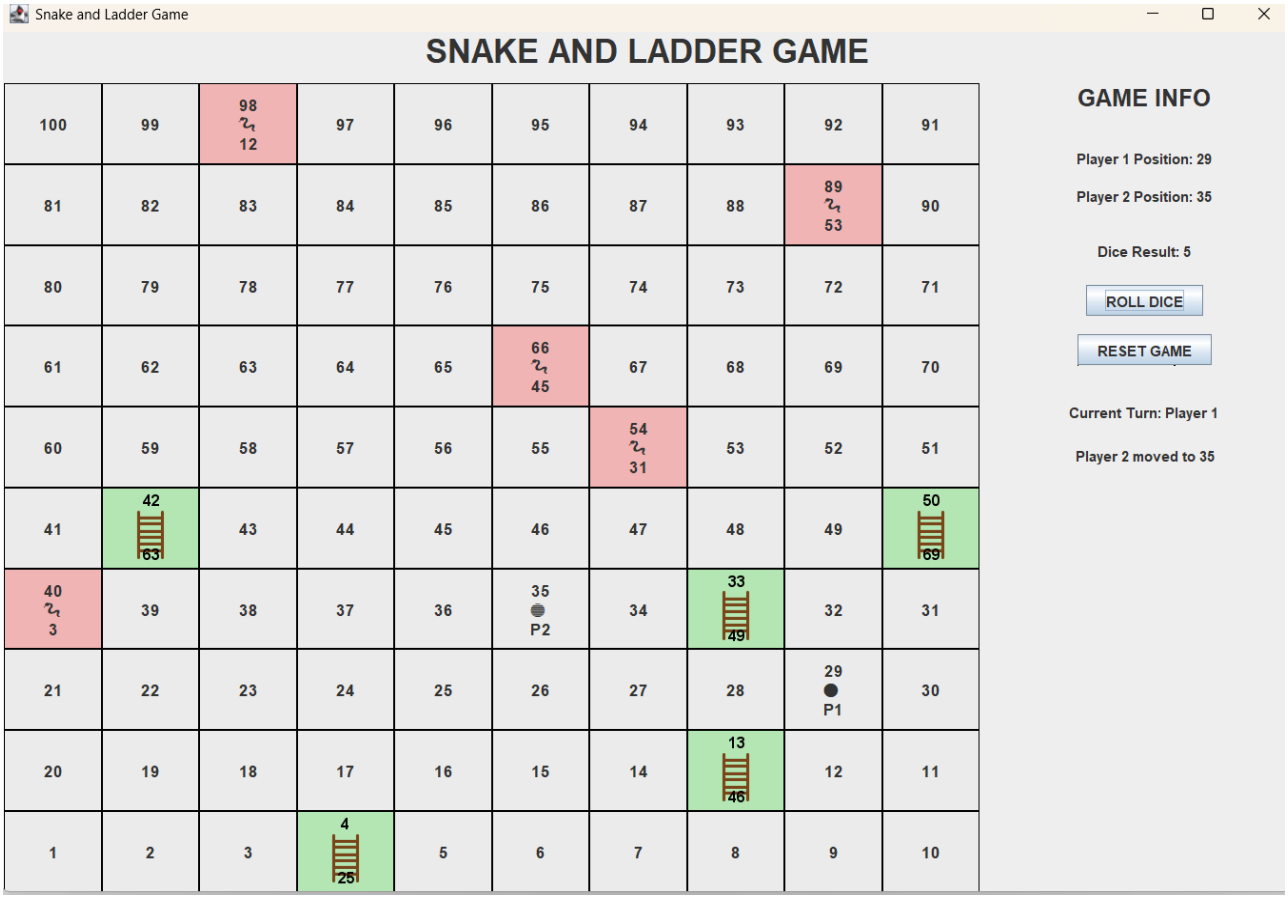
17. Limitations:

- The game supports two players only.
- There is no online multiplayer mode.
- There is no database or permanent score storage.
- The board layout and Snake/Ladder positions are fixed in the code.
- The game does not include advanced movement animation.
- Sound effects and background music are not included.
- The application is designed as a desktop Java application.

18. Future Improvements

- Add animated token movement between cells.
- Use custom graphical Snake and Ladder images instead of text symbols.
- Add sound effects for dice, Snake and Ladder events.
- Add different board layouts and difficulty levels.
- Allow users to choose player names.
- Add a scoreboard and match history.
- Add a single-player mode against a computer-controlled player.
- Add save/load game functionality.
- Improve the visual theme and add background graphics.

19. Result and Discussion:



The Snake and Ladder Game was implemented as a Java Swing desktop application. The final system provides a 10×10 graphical board, two-player turn management, random dice generation, visible player positions, Snake and Ladder movement, reset functionality and winning detection. The project demonstrates how GUI components, event handling, Object-Oriented Programming, HashMap-based position mapping and game-state logic can be combined to build an interactive

desktop application. The incremental development process also made it possible to implement and test the game in separate stages.

20. Conclusion:

The project successfully demonstrates the development of a simple interactive Snake and Ladder game using Java. Starting from the basic GUI and board structure, the project was gradually extended with dice functionality, player movement, graphical tokens, Snake and Ladder rules, turn management and the final winning condition. The project provided practical experience with Java Swing, classes and objects, encapsulation, event handling, Random number generation, HashMap and GUI state updates. The final application meets the main requirements of the proposed game and provides a suitable foundation for future improvements such as animation, sound, player customization and advanced graphics.